

COMP 597 Final: Contrastive Learning using SimCLR and ProxyNCA

1st Ryan Chang

School of Science, Engineering, and Technology
The Pennsylvania State University - Harrisburg
Harrisburg, USA
rjc5887@psu.edu

2nd Joel Horne

School of Science, Engineering, and Technology
The Pennsylvania State University - Harrisburg
Harrisburg, USA
jth6046@psu.edu

3rd Haytham Zaami

School of Science, Engineering, and Technology
The Pennsylvania State University - Harrisburg
Harrisburg, USA
hmz5330@psu.edu

Abstract—Contrastive learning provides a supervising method for learning image relations by finding cosine similarities between different transformations of the same image. Two different proven contrastive learning implementations, SimCLR using NT-Xent loss and ProxyNCA, are compared in this report. We trained models using two different backbones: ResNet-18, and ResNet-50 using both random unfrozen weights and weights from ImageNet. We found that ProxyNCA outperforms basic SimCLR by a wide margin.

Index Terms—Machine Learning, Computer Vision, Contrastive Learning, SimCLR, NT-Xent, ProxyNCA, Recall@k, Transfer Learning, CUB2011, ResNet

I. INTRODUCTION

While curated data is often labeled, many real-world datasets may cause labeling to become impractical due to their size or inaccessibility. Contrastive Learning provides a method for machine learning models to learn relationships between images without the need for labels. By contrasting differing views of the same image with batches of images in the dataset, SimCLR models can learn strong feature relationships between images in their embedding space. This can be compared against ProxyNCA which requires labels to train its proxies. In our experiment, we used Contrastive Learning to create relationships between images of 200 different species of birds without relying on their labels.

SimCLR was introduced in 2020 by a team at Google Research led by Ting Chen [1]. It uses a self-supervising contrastive learning algorithm to create image representations without relying on labeled data. The algorithm creates two sets of randomly transformed views of the same image. These views are compared, and the loss between them is calculated using cosine similarity. The contrastive prediction is determined by comparing the losses of the samples in each batch to each other in the projection head. The goal of this method is to create close relationships between images with similar features and more distant relationships between images with dissimilar features. By relying entirely on comparisons, the SimCLR

model does not need labeled data to create relations. This model made significant improvements over previous models on the Imagenet dataset.

ProxyNCA was introduced in 2017 by a Google Research team led by Yair Movshovitz-Attias [2]. This method uses supervised contrastive learning with a true class label for every proxy. The proxies are initialized randomly, but are learnable vectors. Images are then trained to separate from one another by their textures, edges, and defining features. In order to compare images, we use a loss that uses the Euclidean distance between an image, its true class proxy, and a summation of all other proxies. The goal of ProxyNCA is to learn an embedding space where images of the same class cluster tightly around their shared proxy, while being pushed away from proxies of other classes. Unlike sample-based methods such as triplet loss, which require constructing pairs or triplets of images and suffer from polynomial complexity, ProxyNCA reduces the comparison to just K proxies. Because every image only needs to be compared against a fixed set of proxies rather than all other samples in the batch, ProxyNCA scales much better to large datasets.

II. METHODOLOGY

A. Dataset

For this project, we selected the CUB-200-2011 dataset provided by USDC and Caltech [3]. This dataset contains 11,788 images of 200 distinct bird species. Each of these distinct bird species has an associated Wikipedia article and is organized by scientific classification such as order, family, genus, and species. Each image in the dataset has additional related information such as bounding boxes, part locations, and attributes. However, due to the nature of contrastive learning and its use of self supervision, we ignored the related information and had the model learn to embed solely from the images in the dataset. The dataset is split into 5,994 training images and 5,974 testing images.

B. Model Comparison

Our experiment compared two different backbones, ResNet-18 and ResNet-50. These acted as the model encoder for both SimCLR and ProxyNCA. As prescribed by SimCLR’s method, we continually trained the projection head, but we also tested each backbone under the condition of whether their parameters had their gradients open to being modified (unfrozen) or not (frozen). For ProxyNCA we tested each backbone with frozen and unfrozen ImageNet weights, and for SimCLR we only used frozen weights on backbones using the ImageNet weights.

1) *SimCLR*: SimCLR stands for Simple Framework for Contrastive Learning of Visual Representations. The SimCLR model uses four components to create contrastive relations (Fig. 1). The *data augmentation* module creates two views using a stochastic transformation pipeline. These views are fed into the *base encoder*, which extracts representation tensors from them. The original study uses ResNet as their base encoder, which we replicate. These representation tensors are then mapped to a 128-dimensional space by a *projection head*, where then contrastive loss is applied. This is the portion of the model that undergoes training. Loss is then calculated using a *contrastive loss function* that quantifies the similarity between the created views proportional to the sum of the similarities between the one created view and the views of all the other examples in a selected mini-batch (sampled subset of the batch) of N example views. This is formally defined as such:

$$L_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)}$$

Where z_i and z_j are the created views, $\text{sim}(u, v)$ is the cosine similarity between the two views, $\mathbf{1}_{[k \neq i]} \in \{0, 1\}$ evaluates to 1 if and only if $k \neq i$ (preventing the compared views in the denominator from being equal for accurate batchwise negative contrast) and τ is a temperature parameter. The N examples in the sampled mini-batch act as negative examples when compared to the positive pair (z_i, z_j) . This loss function, called Normalized Temperature-Scaled Cross Entropy Loss (NT-Xent) ensures the creation of an embedding space that groups views with similar features together and those with contrasting features further apart.

2) *ProxyNCA*: ProxyNCA stands for Proxy Neighborhood Component Analysis. It is built on the idea of comparing images to proxies, rather than images to each other. The reasoning behind this is to make training more computationally efficient, by clustering non-labeled images to a single proxy. Fig. 2 breaks down how images are grouped to their proxies. Each diamond acts as a proxy, where images are clustered around their true class.

Each class has a proxy, which is a learnable vector in the embeddings. During training, we used the backbone *ResNet* with 18 and 50 layers, respectively. During the initial implementation, we trained on frozen and unfrozen backbones. The goal of training ProxyNCA is to adjust those weights,

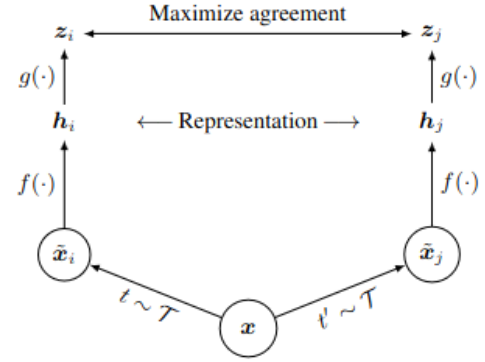


Fig. 1. SimCLR Overview: x is the original image, $\hat{x}_i$ and $\hat{x}_j$ are the contrasting views of x , $f(\cdot)$ encodes the representations h_i, h_j , $g(\cdot)$ projects the representations into 128D space, and finally NT-Xent loss is determined between the encoded z_i and z_j , which is used to train the encoder (Image Source: [1])

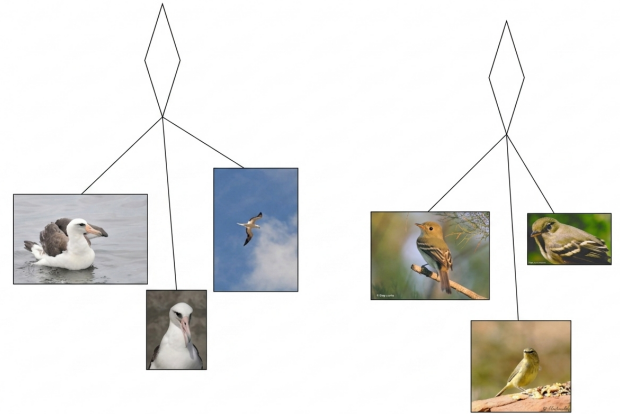


Fig. 2. How images are trained with Proxies (Image Source: CUB-2011)

so freezing the models would not help the model’s accuracy. This is something we highlight in the results section. In ProxyNCA’s forward pass, a batch of images is fed through the backbone to produce embeddings. From here, the Euclidean distance is computed between each image and every proxy. This is then used in the loss function, which is defined as:

$$L_{NCA}(x, y, Z) = -\log\left(\frac{\exp(-d(x, y))}{\sum_{z \in Z} \exp(-d(x, z))}\right)$$

Where d computes the Euclidean distance between points, x is the embedding of sample x , y is the true proxy (what class x belongs to), and Z , which is the set of all proxies. During backpropagation, the backbone and proxies are updated according to the gradient produced by the loss. The gradient moves the proxies closer to the cluster of their true class until the loss is minimized.

C. Training

For this project, we started with a single Jupyter notebook, but in order to train efficiently and obtain all the results within the project time frame, we branched out and trained using our

own scripts on different hardware. At the end we consolidated all of our code into one notebook for the final submission.

1) *SimCLR*: Our SimCLR models’ training script was adapted from “Contrastive Learning with SimCLR in PyTorch” by the GeeksForGeeks team [4]. We trained two baseline models using ResNet-18 and ResNet-50 as the backbones. Both base models were trained with a learning rate of $\alpha = 1e^{-4}$ and a batch size of 512. Due to VRAM limitations on the hardware used to train the models, batch accumulation methods were implemented to simulate 512 batches during the training process. For the SimCLR models, a randomized set of image transformations including crops, horizontal flips, color jitter, grayscale, and Gaussian blur are applied to each sample image and converts the resultant views to tensors. Each base model was trained for 500 epochs, and intermediate models were saved every 50 epochs to record loss and evaluate Recall@k at each milestone. The Adam optimizer was used to train these models to minimize the NT-Xent loss function.

2) *SimCLR - Transfer Learning*: We decided to try training a SimCLR model using transfer learning and pretrained model encodings. For this experiment, we trained two models and once again used a backbone of ResNet-18 and ResNet-50, and we used IMAGENET1k_V1 and IMAGENET1k_V2 for the pretrained weights of the ResNet-18 and ResNet-50 transfer learning models respectively. During the training, we froze the pretrained IMAGENET1k weights and only trained the projection head for each model. We decided to use a slightly higher learning rate of $\alpha = 5e^{-3}$ so that the projection head could learn more quickly and strengthen the confidence of the pretrained ImageNet weights to classify our dataset. Batch accumulation was also used during the transfer learning training runs to simulate a larger batch size of 512. These transfer learning models were trained for 10 epochs. The Adam optimizer was also used during the transfer learning process to apply gradient descent and minimize the NT-Xent loss similar to the base models. The transfer learning models were saved at each epoch to record loss and evaluate Recall@k at each milestone.

3) *ProxyNCA - Initial and adjusted implementation*: During the training phase, we initially wanted to implement zero-shot learning, where the trained ProxyNCA embeddings would be trained on a set of classes, and evaluated on a different set of classes the model had never seen before. This was inspired by *PyTorch Implementation of ProxyNCA* [5], by Tschernezki et. al. The goal of zero-shot learning is to further reinforce the idea of embeddings not being used post-training. However, the CUB-200-2011 dataset being used in our implementation already defines a training and testing split for the images. When the test dataset and training dataset are split and filtered to have non-overlapping classes, the training and testing datasets were far too small. This resulted in losses and Recall@k being lower than anticipated. In response, we decided to switch to the traditional training and testing groups, having a near even split for training and testing. By doing this, we were able to train and test on more images, which led to better results. For the ProxyNCA models, random transformations

such as resizing, random cropping, and random horizontal flipping are applied to each sample image during training. This contrasts to SimCLR’s transformation function which had more transformations applied to the images.

One of the key points to training the ProxyNCA model is the idea of using unfrozen weights in order to have better embeddings. During our training of ProxyNCA, we implemented a total of four models trained on 50 epoch each; ResNet-18 with frozen weights, ResNet-18 without frozen weights, ResNet-50 with frozen weights, and ResNet-50 without frozen weights. The goal of ProxyNCA is to adjust these weights during training, so having frozen weights defeats the purpose of training with proxies. This can be noted in our results, where unfrozen ResNet models performed much better in comparison to the frozen models. All models were trained using the same learning rates; the encoder uses $\alpha = 1e^{-4}$ and the proxies used a learning rate of $\alpha = 1.0$. Similar to the SimCLR models, the Adam optimizer was used to train all four of the ProxyNCA models to minimize loss.

Our training and evaluation scripts may be found at our project repository [6].

D. Evaluation Method

To evaluate our models, we used Recall@k. This method involves creating an embedded image space using the test data set. This method utilizes the k-Nearest Neighbors algorithm to find the nearest k neighbors of each sample image in the test dataset, excluding itself. Once all neighbors are found, Recall@k checks the class labels of the nearest neighbors, and if at least one of the nearest neighbors has the same class label as the sample test image, then this indicates that the model has correctly classified the bird species in the image. The final accuracy is calculated using the equation:

$$\text{accuracy} = \frac{\text{correctly identified samples}}{\text{total test samples}}$$

We evaluated our models using Recall@k with a value of $k = 1, 3, 5, 10$. Recall@1 indicates how well our models actually learned to embed images of the same class nearby in the embedding space because the accuracy of the model is determined solely by the nearest neighbor of the test sample. As we increase k, if we observe that the recall accuracy improves, we can confirm that images of similar bird species are generally close together in the embedded image space.

III. RESULTS

A. SimCLR

After 500 epochs of training, the base ResNet-18 model without transfer learning was able to minimize loss from 1.3404 to 0.3363 (Fig. 3). The base ResNet-18 model’s Recall@1 improved from 6.14% to 7.75% over 500 epochs, while Recall@10 increased from 28.22% to 32.05% (Fig. 4).

The base ResNet-50 model minimized loss from 1.6455 to 0.3199 (Fig. 5). The Recall@1 for the base ResNet-50 improved from 4.82% to 8.34% over 500 epochs, and Recall@10 improved from 25.53% to 32.84% (Fig. 6). We did not see much improvement for Recall@k accuracy the

more epochs we trained, and we believe that the base models started to overfit to the training data and as a result were not able to generalize well for the evaluation on the test dataset.

These results are what led to our decision to try and train our SimCLR models by taking pretrained weights and applying transfer learning methods by freezing the pretrained weights and only training the projection head. As a result, after training for just 10 epochs, the ResNet-18 model was able to minimize loss from 5.2965 to 2.2628 (Fig. 7). The Recall@1 for the ResNet-18 transfer learning model improved to 18.42% accuracy, which was a noticeable improvement from the base model's 7.75% accuracy. And Recall@10 for this model was able to achieve 57.8% (Fig. 8), which almost doubled the accuracy from the base ResNet-18 model that we trained.

The ResNet-50 transfer learning model minimized loss from 4.6855 to 1.379 after just 10 epochs (Fig. 9). Through 10 epochs of training, the Recall@1 for this model improved to 29.22% accuracy, and the Recall@10 managed to achieve 71.49% accuracy (Fig. 10). From the results, it's clear that the ResNet-50 transfer learning model had the largest improvement during the evaluation phase and became our best scoring SimCLR model.

B. ProxyNCA

After training each type of ProxyNCA model for 50 epochs, our unfrozen ResNet-18 and ResNet-50 models were able to finish with Recall@10 of 86.50% accuracy and 91.34% accuracy respectively. By adding more layers to our backbone, we were able to have a jump of 3.86% when it came to our Recall@10. As for Recall@1, the unfrozen models performed at 63.548% accuracy for ResNet-18 unfrozen and 75.129% accuracy for ResNet-50 unfrozen. The accuracy of the frozen models fell shorter with a Recall@10 of 78.495% accuracy and 77.252% accuracy for the ResNet-18 and ResNet-50 backbones. Additionally, the Recall@1 of the frozen models only achieved values of 42.665% and 42.32% for ResNet-18 and ResNet-50 backbones respectively.

Our best model was trained with the ResNet-50 backbone without freezing any weights, which produced a Recall@10 of 91.336%. This is in line with our predictions, as Tschernezi et al's Recall@8 using ProxyNCA was 84.52% accuracy [5]. Having enough layers for our backbone and a large enough epoch count allowed us to reach this result.

Figures 12, 13, 14, and 15 visualize how our loss changes over time. For the unfrozen ResNet-50 model, convergence begins to happen around epoch 43, before finally reaching a loss of 1.3596. This beat out our ResNet-18 unfrozen model which was at 1.4098 and was close to half of the frozen models' losses.

IV. CONCLUSION

From our results, we can see that the unfrozen ResNet-50 ProxyNCA model achieved superior classification capabilities compared to all SimCLR models and all other ProxyNCA models that were trained throughout this project. The ResNet-50 ProxyNCA model's Recall@1 accuracy obtained better

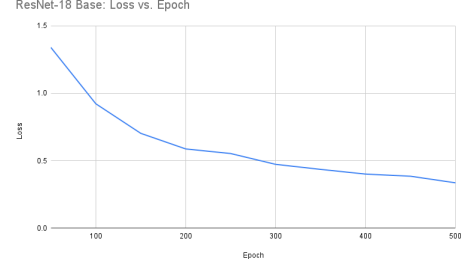


Fig. 3. SimCLR: ResNet-18 Base Model Loss vs Epoch

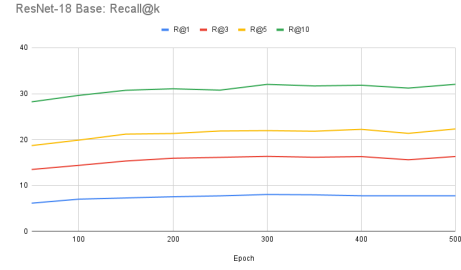


Fig. 4. SimCLR: ResNet-18 Base Model Recall@k vs Epoch

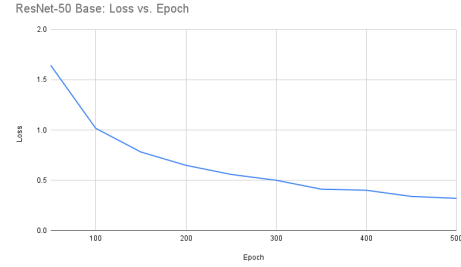


Fig. 5. SimCLR: ResNet-50 Base Model Loss vs Epoch

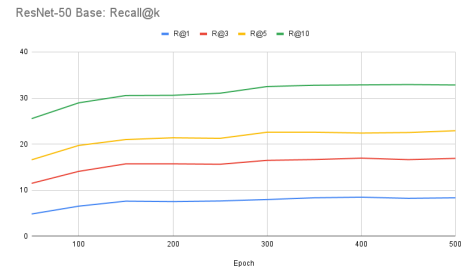


Fig. 6. SimCLR: ResNet-50 Base Model Recall@k vs Epoch

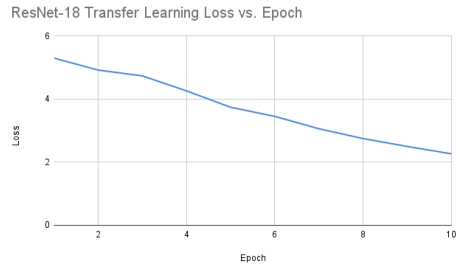


Fig. 7. SimCLR: ResNet-18 Transfer Learning Model Loss vs Epoch

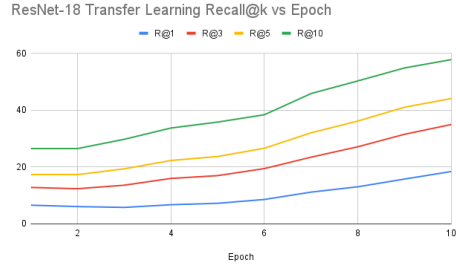


Fig. 8. SimCLR: ResNet-18 Transfer Learning Model Recall@k vs Epoch

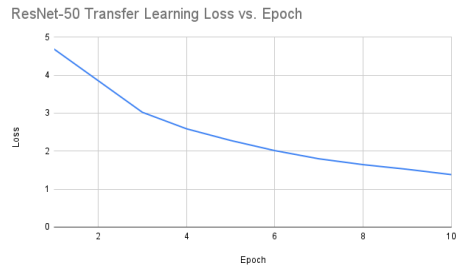


Fig. 9. SimCLR: ResNet-50 Transfer Learning Model Loss vs Epoch

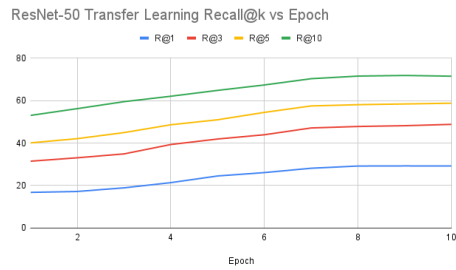


Fig. 10. SimCLR: ResNet-50 Transfer Learning Model Recall@k vs Epoch

Model	R@1	R@3	R@5	R@10
ResNet18	7.75	16.31	22.32	32.05
ResNet50	8.34	16.9	22.9	32.84
Frozen ResNet18	18.42	34.97	44.11	57.8
Frozen ResNet50	29.22	48.81	58.77	71.49

Fig. 11. SimCLR: Recall@k values for final models

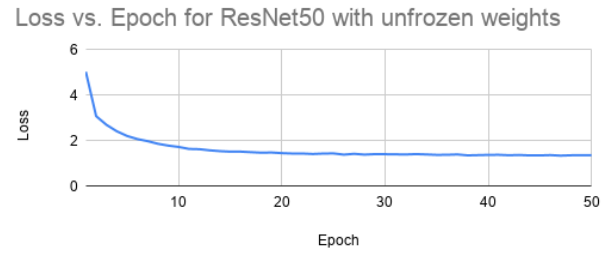


Fig. 12. ProxyNCA Loss graph over 50 epochs for unfrozen ResNet-50

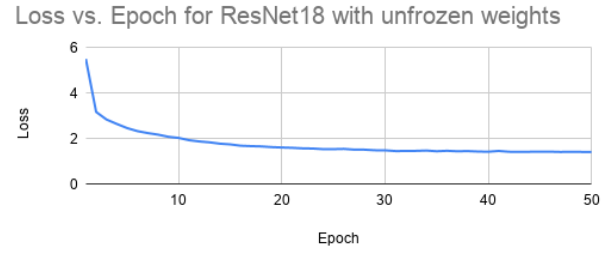


Fig. 13. ProxyNCA Loss graph over 50 epochs for unfrozen ResNet-18

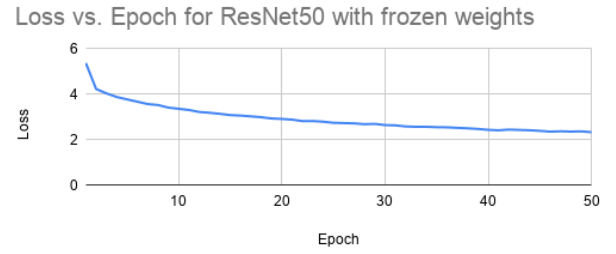


Fig. 14. ProxyNCA Loss graph over 50 epochs with frozen ResNet-50

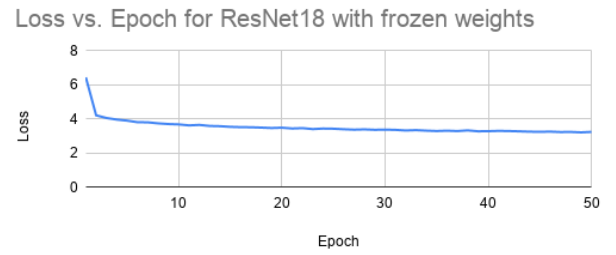


Fig. 15. ProxyNCA Loss graph over 50 epochs with frozen ResNet-18

Model	R@1	R@3	R@5	R@10
ResNet18	63.548	78.046	82.827	86.503
ResNet50	75.129	85.46	88.971	91.336
Frozen ResNet18	42.665	63.152	71.143	78.495
Frozen ResNet50	42.32	61.892	70.711	77.262

Fig. 16. Trained ProxyNCA models' Recall@K values

results than the best SimCLR model’s Recall@10 accuracy. We believe that the unfrozen ResNet-50 ProxyNCA model performed better than the base SimCLR models due to not starting with random weights. Instead, the ProxyNCA model strengthens the confidence of the existing pretrained model weights through the use of proxies during training. In a way, this could be considered a method of fine tuning the model itself, which would help to explain why freezing the pretrained model weights during ProxyNCA produced worse results. This also explains why ProxyNCA requires a trainable backbone. By freezing the weights, the model is not able to learn how to classify images from the proxies. Instead, we would just be evaluating ResNet’s backbones and how well they are able to identify images from the CUB-2011 dataset.

In conclusion, we believe that our results indicate that the use of proxies in ProxyNCA is the superior implementation of contrastive learning and provides a better method for self supervised learning for image classification. Proxies are trainable vectors, thus allowing them to change over time to create space between different image clusters. This allows for features to be more strongly attracted to the closest proxy, which allows for similar images to be grouped more tightly together in the embedding space. Because the ProxyNCA model must use labels to identify true class proxies when training, there is a tradeoff between recall accuracy and requiring labels during training, which is observed in the performance contrast between SimCLR and ProxyNCA.

V. FUTURE IMPROVEMENTS

For future extensions to this project we would want to explore using additional backbones such as ResNet-101 or CLIP. There were several attempts made to train models using a ResNet-101 backbone, however most attempts to load and train the ImageNet weights were met with memory errors. This may either be an issue with our code, or the ImageNet weights may cause the ResNet models to grow to a size that cannot fit in the hardware we tested on. In addition we would want to explore different loss functions such as ProxyAnchor to see if we can improve the model even further [7].

We also explored linear classification using the contrastive learning image transforms. A single linear layer was trained as a classifier on the dataset used in our experiment by replacing the projection head in the SimCLR model with the linear layer. Two different models were trained: the first model augmented the dataset with the simple evaluation transform, and the second with SimCLR’s stochastic edit training transform. We expected the model trained on the stochastic transform to perform worse, as the views produced by the transform would have fewer distinguishable features, and generally less consistency. Not surprisingly, this is what we observed. The model trained on the simple evaluation transform achieved a training accuracy of 88.87% and a validation accuracy of 57.21%, while the stochastic transform-trained model achieved on 47.81% training and 33.97% validation. This serves as a great example of "garbage-in, garbage-out": image data that did not survive "corruption" by the transform meant for

contrastive learning did not yield effective data for training the classifier. This model could be tested further by training it using the updated ProxyNCA model’s backbones, or for more epochs.

VI. CONTRIBUTIONS

Ryan Chang: SimCLR Training, Transfer Learning Training, Recall@k evaluation script, Gathering data results for report, Batch Accumulation implementation

Joel Horne: Training and Evaluation code writing and formatting, Linear Classifier, Transfer Learning Training

Haytham Zaami: ProxyNCA Training and Evaluation, Gathering data results for report

REFERENCES

- [1] T. Chen, S. Kornblith, M. Norouzi, and G. E. Hinton, "A simple framework for contrastive learning of visual representations," *CoRR*, vol. abs/2002.05709, 2020. [Online]. Available: <https://arxiv.org/abs/2002.05709>
- [2] Y. Movshovitz-Attias, A. Toshev, T. K. Leung, S. Ioffe, and S. Singh, "No fuss distance metric learning using proxies," 2017. [Online]. Available: <https://arxiv.org/abs/1703.07464>
- [3] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, "Perona lab - cub 200-2011," California Institute of Technology, Tech. Rep. CNS-TR-2011-001, 2011.
- [4] GeeksForGeeks. (2025) Contrastive learning with simclr in pytorch. Online. Accessed: Apr 30, 2026. [Online]. Available: <https://www.geeksforgeeks.org/deep-learning/contrastive-learning-with-simclr-in-pytorch/>
- [5] V. Tschernezki, A. Sanakoyeu, and B. Ommer, "Pytorch implementation of proxynca," <https://github.com/dichotomies/proxy-nca>, 2020.
- [6] R. Chang, J. Horne, and H. Zaami, "Comp597 computer vision final project - spring 2026," Online, 2026, accessed: Apr 30, 2026. [Online]. Available: <https://github.com/jhorne98/comp597-computer-vision-final>
- [7] S. Kim, D. Kim, M. Cho, and S. Kwak, "Proxy anchor loss for deep metric learning," June 2020.